Unity Catalog In Databricks

What Is the Unity Catalog in Databricks?

Unity Catalog is a **unified governance and data security layer** for Databricks that provides centralized access control, data lineage, and auditing across **all workspaces and data assets** (tables, files, machine learning models, dashboards).

It simplifies governance by bringing **fine-grained access management** to **structured, semi-structured, and unstructured data** in the **lakehouse architecture**.

Key Features of Databricks Unity Catalog

1. Centralized Governance

A single **metastore** manages all data assets across multiple workspaces.
This ensures consistency in policies, permissions, and compliance.

2. Fine-Grained Access Control

Provides **table, row, and column-level security** for sensitive data.
Access is enforced using standard **ANSI SQL GRANT/REVOKE** statements.

3. Data Lineage Tracking

Automatically tracks how data flows from **ingestion to dashboards**.
Helps with **impact analysis, debugging, and compliance audits**.

4. Audit Logging

Captures detailed logs of **who accessed what data and when**.
Essential for meeting **regulatory and security requirements**.

5. Multi-Cloud Support

Works seamlessly across **AWS, Azure, and GCP** environments.
Enables a unified governance strategy in a **multi-cloud setup**.

6. Integration with BI Tools

Supports direct connections with **Power BI, Tableau, Looker**, and others.
Allows analysts to **query governed data securely**.

7. Support for External Locations

Lets you register and manage **external storage locations**.
Provides governance over data even if it's outside managed tables.

Benefits of Databricks Unity Catalog

1. Consistency

Provides a **single governance model** across all workspaces.
Reduces duplication of access policies and simplifies management.

2. Security

Enables **centralized fine-grained access control** for sensitive data.
Helps organizations stay **compliant with standards like GDPR, HIPAA**.

3. Productivity

Allows users to **self-serve data** without waiting for IT.
Speeds up analytics and ML workflows with governed access.

4. Flexibility

Supports **structured, semi-structured, and unstructured data**.
Works seamlessly with **SQL, Python, R, Scala, and ML workloads**.

5. Scalability

Designed for **petabyte-scale data management** in the lakehouse.
Ensures governance remains effective as data volume grows.

Unity Catalog Architecture

Unity Catalog acts as a **single governance and security layer** on top of the Databricks Lakehouse, ensuring consistent access control and lineage tracking across all workspaces.

1. Metastore

The **root governance container**, created per **region and account**.

Stores metadata and access policies for all catalogs, schemas, and tables.

2. Catalog

A **top-level namespace** that groups schemas, typically by **business domain** (e.g., finance_catalog).

Helps organize data assets logically and simplifies governance at the domain level.

3. Schema (Database)

Organizes **tables and views** within a catalog.

Acts like a traditional database schema, making it easier to **separate projects or teams**.

4. Tables & Views

The actual **data assets** managed under Unity Catalog.

Tables can be **managed (Databricks controls storage)** or **external (linked to cloud storage)**, while views provide virtual access.

5. Permissions

Governed through **ANSI SQL GRANT/REVOKE** statements.

Supports **table, row, and column-level security**, ensuring least-privilege access.

Conceptual Diagram

Metastore

└── Catalogs (e.g., finance_catalog)

└── Schemas (e.g., transactions_schema)

└── Tables/Views (e.g., payments, customers)

Implementation Steps for Databricks Unity Catalog

Prerequisites

Before setting up Unity Catalog, ensure:

- You are on **Databricks Premium or Enterprise plan**.
- You have **Account Admin** role in Databricks.
- A **cloud storage account** is ready (S3 for AWS, ADLS for Azure, GCS for GCP).
- You have required **IAM roles/policies** created for Databricks to access storage.

Step 1 – Create a Unity Catalog Metastore

- Log in to the **Databricks Account Console**.
- Navigate to **Data → Unity Catalog → Metastores**.
- Click **Create Metastore** and provide:
  - **Name**: e.g., primary_metastore.
  - **Region**: Same as your workspace.
  - **Root Storage**: e.g., s3://my-uc-storage/ (AWS) or abfss://container@storage.dfs.core.windows.net/ (Azure).
  - **IAM Role** (cloud-specific).
- Click **Create**.

Step 2 – Assign Metastore to Workspaces

- Open the newly created **Metastore details**.
- Select **Assign to Workspaces**.
- Choose the workspace(s) you want Unity Catalog enabled for.
- Mark as **Default Metastore**.

Step 3 – Create Catalogs, Schemas, and Tables

Go to a **Databricks SQL Warehouse or Notebook** and run:

```
-- Create a catalog
CREATE CATALOG finance_catalog;
-- Create a schema (database)
CREATE SCHEMA finance_catalog.transactions_schema
COMMENT 'Stores all transaction-related tables';
-- Create a managed table
CREATE TABLE finance_catalog.transactions_schema.payments (
  id INT,
  customer_id STRING,
  amount DECIMAL(10,2),
  payment_date DATE
);
-- Create an external table (optional)
CREATE TABLE finance_catalog.transactions_schema.external_payments
USING PARQUET
LOCATION 's3://raw-data-bucket/finance/payments/';
```

Step 4 – Manage Permissions

Use **ANSI SQL GRANT/REVOKE** statements:

```
-- Grant catalog usage
GRANT USAGE ON CATALOG finance_catalog TO `analyst_group`;
-- Grant schema usage
GRANT USAGE ON SCHEMA finance_catalog.transactions_schema TO
`analyst_group`;
-- Grant read access to a table
```

```
GRANT SELECT ON TABLE
finance_catalog.transactions_schema.payments TO `analyst_group`;

-- Revoke access

REVOKE SELECT ON TABLE
finance_catalog.transactions_schema.payments FROM `analyst_group`;
```

Step 5 – Enable Lineage and Auditing

- Enable **audit logs** in Databricks to track data access.
- Use Unity Catalog's **data lineage** to trace datasets from ingestion to reporting.

Step 6 – Verification

Run the following to check setup:

```
-- List catalogs

SHOW CATALOGS;

-- List schemas in a catalog

SHOW SCHEMAS IN finance_catalog;

-- List tables in a schema

SHOW TABLES IN finance_catalog.transactions_schema;
```